

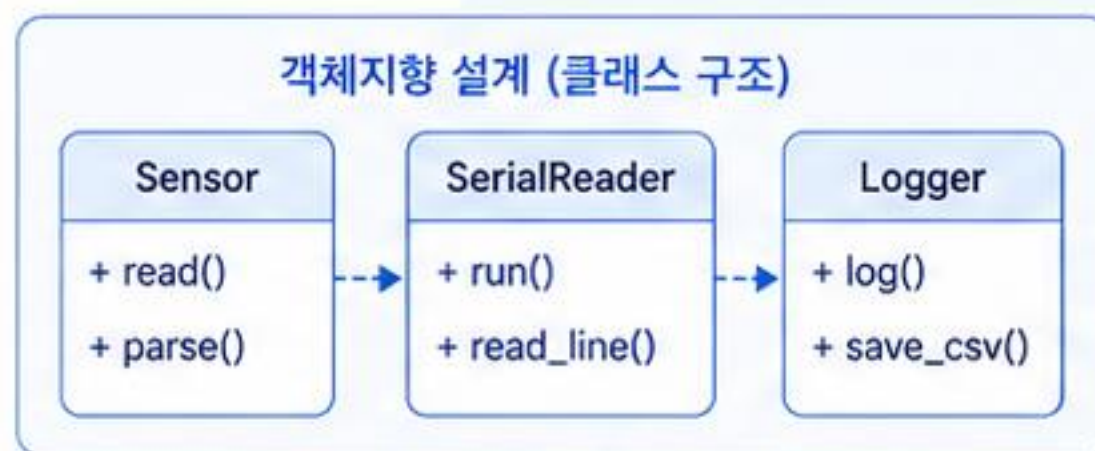
Day 3

객체지향 · 예외처리 · 멀티스레드

센서 수집 시스템을 구조적으로 설계하기

임베디드 · IoT 개발자 과정

- 객체지향 프로그래밍과 클래스 설계
- 예외 처리로 안정적인 코드 작성
- 멀티스레드로 동시 작업 구현
- 센서 데이터를 구조적으로 수집 · 저장 · 분석



예외 처리 예시

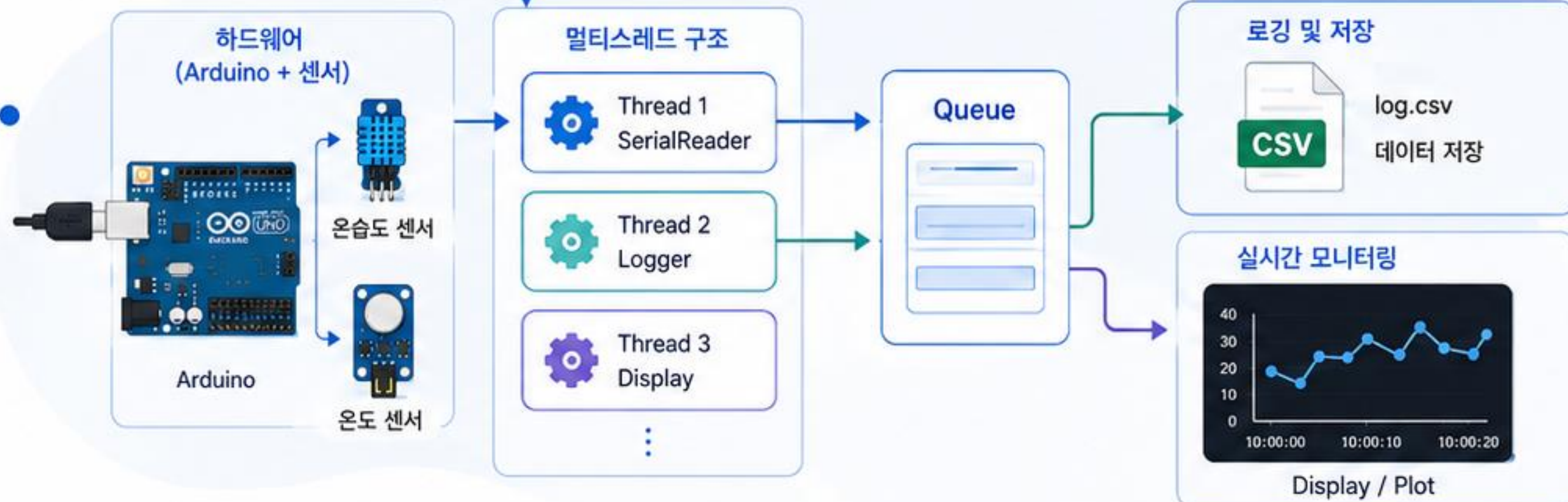
```

try:
    line = ser.readline().decode().strip()
except serial.SerialException as e:
    print(f"Serial Error: {e}")
except Exception as e:
    print(f"Error: {e}")
    
```

터미널 모니터

```

$ python main.py
TEMP:24.5,HUMI:60
TEMP:24.7,HUMI:61
TEMP:24.6,HUMI:59
...
    
```



Day 3

오늘의 학습 목표

Day 3에서 배우게 될 핵심 내용을 한눈에 정리합니다.

- 1  Python 클래스와 객체 이해
- 2  센서 클래스로 코드 구조화
- 3  예외 처리로 안정성 높이기
- 4  Threading으로 동시 작업 처리
- 5  Queue 기반 센서 수집 시스템 구현



객체 설계부터 예외 처리, 멀티스레드 센서 수집까지 단계적으로 학습합니다.

학습 흐름 로드맵

- 1  1. 클래스 / 객체
- 2  2. 센서 클래스 설계
- 3  3. 예외 처리
- 4  4. 멀티스레드
- 5  5. Queue 기반 IoT Logger



실습 프로젝트

센서 데이터 수집 및 저장 시스템 완성

03 Day 2 핵심 복습

어제까지 배운 내용을 간단히 복습하고, 오늘 배울 내용을 연결해 봅시다.

☑ Day 2 학습 내용 체크리스트

- ✓ **모듈(Module)**
.py 파일로 기능을 분리하고 import하여 사용
- ✓ **패키지(Package)**
관련 모듈들을 폴더로 묶어 관리 (__init__.py)
- ✓ **프로젝트 구조**
패키지 기반으로 코드 구조를 체계적으로 구성
- ✓ **pip**
외부 패키지 검색, 설치, 제거 방법
- ✓ **가상환경(venv)**
프로젝트별 독립된 Python 환경 구성
- ✓ **requirements.txt**
의존성 목록을 파일로 관리하고 설치
- ✓ **pyserial 기초**
시리얼 통신을 위한 pyserial 사용법 학습

📁 프로젝트 구조 예시

```
sensor_project/  
├── sensor_project/  
│   ├── __init__.py    # 패키지 초기화  
│   ├── sensor.py      # 센서 관련 모듈  
│   ├── logger.py      # 로깅 관련 모듈  
│   └── utils.py       # 유틸 함수 모듈  
├── main.py            # 메인 프로그램  
├── requirements.txt   # 의존성 목록  
└── README.md         # 프로젝트 설명
```

사용 예시

```
from sensor_project.sensor import Sensor  
from sensor_project.logger import DataLogger
```

💡 핵심 포인트

- import를 사용하여 필요한 기능을 가져다 사용
- 패키지 구조로 코드를 체계적으로 관리
- venv와 requirements.txt로 환경을 재현 가능하게!

☆ 기억해야 할 핵심 명령어

패키지 검색

```
pip search <패키지명>
```

패키지 설치

```
pip install <패키지명>
```

패키지 제거

```
pip uninstall <패키지명>
```

의존성 목록 생성

```
pip freeze > requirements.txt
```

의존성 목록 설치

```
pip install -r requirements.txt
```

가상환경 생성

```
python -m venv .venv
```

가상환경 활성화 (Windows)

```
.venv\Scripts\activate
```

가상환경 활성화 (macOS/Linux)

```
source .venv/bin/activate
```



오늘은 여러 파일로 나눈 코드를 **객체(Object)**와 **스레드(Thread)**로 더 구조화하여,
여러 센서를 동시에 안전하고 효율적으로 수집하는 시스템을 만들어 보겠습니다!

복잡해지는 센서 시스템을 구조적으로 설계하고, 유지보수와 확장을 쉽게 만들 수 있습니다.



Point

객체지향은 '데이터(속성)'와 '기능(메서드)'을 하나로 묶어 관리하는 방법입니다.

⚠ 함수와 전역변수만 사용한 코드 (비구조적)

- ❌ 전역변수가 많아짐
- ❌ 관련 기능이 흩어져 있음
- ❌ 코드 재사용이 어려움
- ❌ 센서가 늘어나면 코드가 복잡해짐
- ❌ 수정 시 다른 부분에 영향을 줄 수 있음

```
# 전역변수
sensor_name = "DHT11"
sensor_value = 0

# 센서 읽기 함수
def read_sensor():
    global sensor_value
    # ... 센서 읽기 코드

# 데이터 저장 함수
def save_data():
    global sensor_value
    # ... 저장 코드

# 화면 출력 함수
def show_data():
    global sensor_value
    # ... 출력 코드
```

✅ 객체지향으로 구조화한 코드 (구조적)

- ✅ 관련 데이터와 기능을 하나로 묶음
- ✅ 코드 재사용이 쉬움
- ✅ 확장과 수정이 쉬움
- ✅ 여러 센서를 객체로 관리 가능
- ✅ 유지보수와 테스트가 쉬움

```
class Sensor:
    def __init__(self, name):
        self.name = name
        self.value = 0

    def read(self):
        # ... 센서 읽기 코드
        pass

    def save(self):
        # ... 저장 코드
        pass

    def show(self):
        # ... 출력 코드
        pass
```

여러 센서를 객체로 관리



temp_sensor
(Sensor 객체)



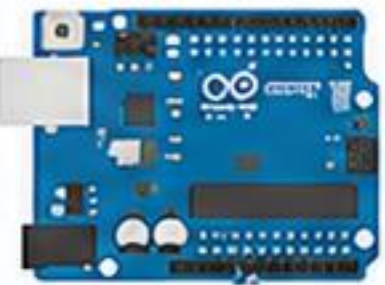
humi_sensor
(Sensor 객체)



light_sensor
(Sensor 객체)

...

Arduino/AVR에서의 모듈화 경험



- ✅ 센서별로 함수로 분리
- ✅ 구조체(struct)로 데이터 관리
- ✅ 파일(.h/.c)로 기능 분리
- ✅ 모듈화의 중요성 경험

Python에서는 객체지향으로 한 단계 더!



클래스로 데이터와 기능을 묶어 관리
상속과 다형성으로 확장성 향상
대규모 시스템을 더 쉽게 설계
테스트와 유지보수가 쉬운 구조

IoT 프로젝트에서의 이점



확장성
새로운 센서나 기능 추가가 쉬움



유지보수성
코드 수정 시 영향 범위가 작음



재사용성
다른 프로젝트에서도 재사용 가능



안정성
구조적인 설계로 버그 감소

☆ 예시 비교

비구조적 코드

100줄의 코드를 수정하려면
전체 코드를 이해해야 함 😞

구조적 코드

Sensor 클래스만 이해하면
수정이 가능! 😊



객체지향은 단순히 문법이 아닌, 문제를 구조적으로 해결하는 사고 방식입니다.

클래스는 설계도(틀), 객체는 설계도로 만든 실제 인스턴스입니다.



Point

클래스는 공통된 속성과 기능을 정의하고,
객체는 그 정의를 바탕으로 실제 값을 갖습니다.

설계도 (클래스)

```
class Sensor:
    def __init__(self, name):
        self.name = name
        self.value = 0

    def read(self):
        # 센서 읽기 코드

    def show(self):
        # 출력 코드
```



실제 인스턴스 (객체)



temp_sensor
(Sensor 객체)



humi_sensor
(Sensor 객체)



light_sensor
(Sensor 객체)

...

객체가 가지는 구성 요소

- 속성(데이터)
 - name, value, unit 등
- 기능(메서드)
 - read(), show(), save() 등



핵심 정리

같은 클래스(설계도)로부터
여러 개의 객체를 만들 수 있고,
각 객체는 서로 다른 데이터를 가질 수 있습니다.

사용 예시

```
temp_sensor = Sensor("TEMP")    # 온도 센서 객체 생성
humi_sensor = Sensor("HUMI")    # 습도 센서 객체 생성
light_sensor = Sensor("LIGHT")  # 조도 센서 객체 생성
```

```
temp_sensor.read()    # 온도 읽기
humi_sensor.read()    # 습도 읽기
light_sensor.show()   # 조도 출력
```


Day 3 06 class 기본 문법

클래스 정의

```
class Sensor:
    def __init__(self, name):
        self.name = name
        self.value = 0
    def read(self):
        pass
    def show(self):
        print(self.name, self.value)
```

1 클래스명
2 생성자
3 속성
4 메서드
5 self

1 클래스명
클래스의 이름

2 생성자
객체 생성 시
자동 호출

3 속성
객체가 가지는
데이터

4 메서드
객체의 동작
(함수)

5 self
현재 객체 자신을
가리키는 참조

객체 생성과 사용

```
sensor = Sensor("DHT11")
sensor.read()
sensor.show()
```

→ 1 객체 생성
→ 2 메서드 호출
→ 3 클래스를 통해
실제 인스턴스를 만든다

self

self

self는 현재 객체(자신) 그 자체를 의미합니다.
객체의 속성과 메서드에 접근할 때
self를 사용합니다.

핵심 정리

1 클래스는 속성과 메서드를 묶어
하나의 타입처럼 정의합니다.

2 객체는 클래스로부터
생성된 실제 인스턴스입니다.

3 self는 현재 객체 자신을
가리킵니다.

07 self와 인스턴스 변수

self는 각 객체가 자신의 값을 가지도록 합니다.



Point

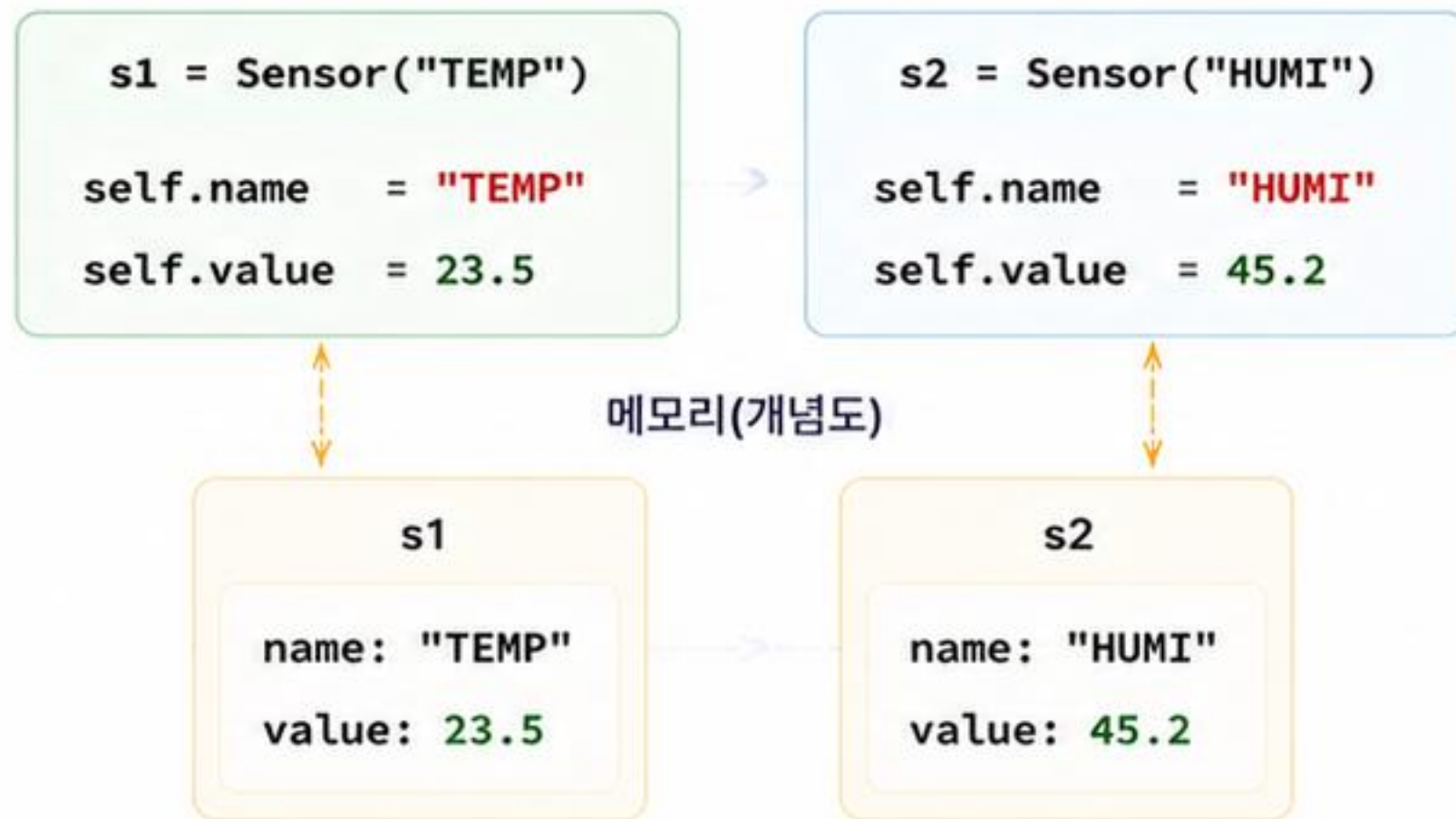
self는 '현재 객체 자신'을 가리키는 참조입니다.
인스턴스 변수는 객체마다 서로 다른 값을 가집니다.

self란?

- self는 메서드 내부에서
현재 객체 자체를 가리키는 이름
- self를 통해 객체의 속성(변수)과
다른 메서드에 접근
- 같은 클래스라도 객체마다
다른 값을 가질 수 있게 해 줌

```
def read(self):  
    # self.value 는 이 객체의 값  
    self.value = 25.3
```

여러 개의 인스턴스(객체)



메모리(개념도)

💡 C 언어의 구조체(struct) 멤버 변수와 유사하지만,
Python에서는 더 쉽고 안전하게 사용할 수 있습니다.

예제 코드

```
class Sensor:  
    def __init__(self, name):  
        self.name = name  
        self.value = 0  
  
    def read(self):  
        # 센서 값을 읽어 저장  
        self.value = 25.0
```

사용 예시

```
s1 = Sensor("TEMP")  
s2 = Sensor("HUMI")  
  
s1.read()  
s2.read()  
  
print(s1.name, s1.value)    # TEMP 25.0  
print(s2.name, s2.value)    # HUMI 25.0
```

C 구조체와 비교



```
struct Sensor {  
    char name[10];  
    float value;  
};
```

- C 구조체는 데이터만 묶음
- Python 클래스는 데이터(속성)와
기능(메서드)을 함께 묶습니다!

핵심 정리

- 1 self는 현재 객체 자신
메서드가 자신을 참조하기 위해 사용
- 2 인스턴스 변수
객체마다 독립적인 값을 가짐
- 3 같은 클래스, 다른 값
여러 객체가 서로 다른 데이터를 가짐



08 생성자 __init__

객체가 생성될 때 자동으로 호출되어 초기값을 설정하는 특별한 메서드입니다.

클래스 정의 예시

```
class Sensor:
    def __init__(self, name, unit):
        # 생성자(생성될 때 1번 호출)
        self.name = name    # 센서 이름 저장
        self.unit = unit    # 단위 저장
        self.value = 0      # 초기 값 설정

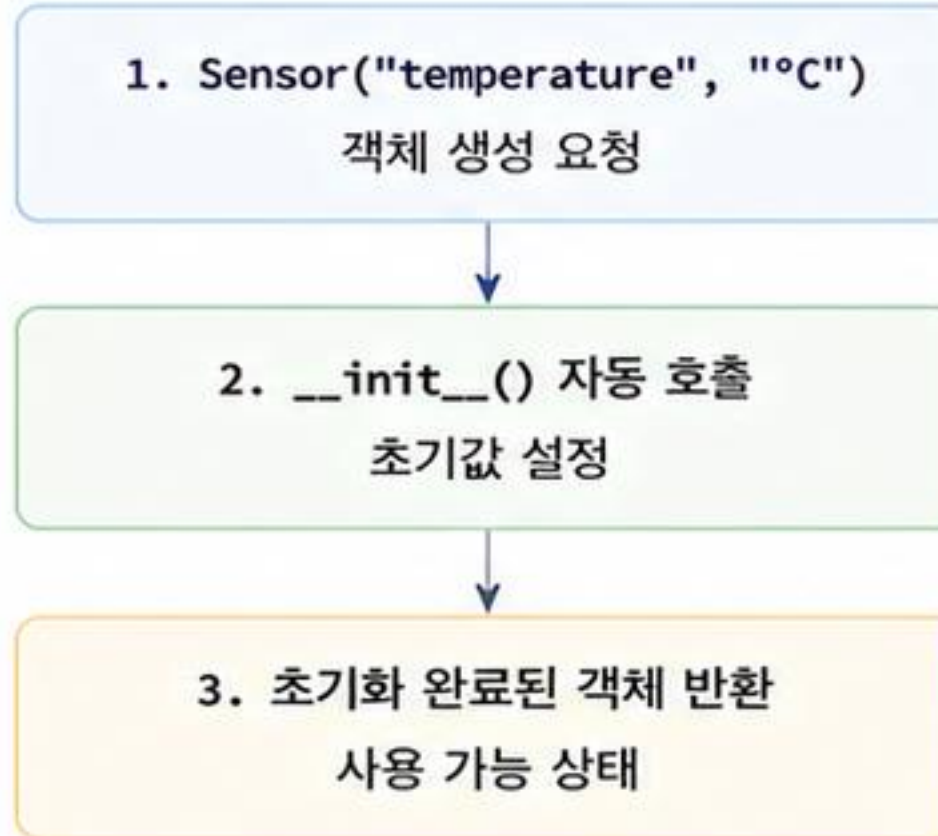
    def read(self):
        # 센서 읽기 기능
        pass
```



핵심 정리

- `__init__(self, ...)`는 생성자(Constructor)입니다.
- `self`는 객체 자신을 의미합니다.
- 객체가 생성될 때 딱 한 번 자동으로 호출됩니다.
- 초기값이 필요한 속성은 `__init__`에서 설정합니다.

객체 생성 과정



Point

`__init__`은 클래스 이름으로 객체를 만들 때 자동으로 호출됩니다.
객체의 초기 상태를 설정하는 역할을 합니다.

작성 예시

```
t = Sensor("temperature", "°C")
h = Sensor("humidity", "%")
```

객체 내부 상태

객체	name	unit	value
t	temperature	°C	0
h	humidity	%	0

다양한 객체 생성 예시

```
temp_sensor = Sensor("TEMP", "°C")
# 온도 센서 객체
```

```
humi_sensor = Sensor("HUMI", "%")
# 습도 센서 객체
```

```
light_sensor = Sensor("LIGHT", "lux")
# 조도 센서 객체
```



생성자를 잘 설계하면, 객체를 **일관되고 안전한 상태**로 시작할 수 있습니다.

Day 3 09 센서 클래스 만들기

IoT 시스템에서 사용할 기본 센서 클래스를 만들어 봅시다.

기본 코드 예시

```
class Sensor:
    def __init__(self, name, unit):
        self.name = name
        self.unit = unit
        self.value = 0

    def read(self):
        # 센서 값 읽기 예시
        self.value = round(random.uniform(20, 30), 2)
        return self.value

    def show(self):
        print(f"{self.name}: {self.value} {self.unit}")

    def to_dict(self):
        return {"name": self.name, "value": self.value, "unit": self.unit}
```

속성 (name, unit, value)

메서드 (read)

메서드 (show)

메서드 (to_dict)



핵심 포인트

- Sensor 클래스 하나로 여러 종류의 센서를 표현 가능
- 속성과 메서드를 함께 묶어 관리
- to_dict()로 저장/전송에 활용 가능

센서 클래스 설계



속성(Attributes)

- name : 센서 이름 (str)
- unit : 단위 (str)
- value : 센서 값 (float)



메서드(Methods)

- read() : 센서 값 읽기
- show() : 값 출력
- to_dict() : dict로 변환



사용 예시

```
t = Sensor("Temp", "°C")
t.read()
t.show()
```

출력 결과

Temp: 24.31 °C



온도 센서
Temp
(°C)



습도 센서
Humi
(%)



조도 센서
Light
(lux)



실습 미션

- 1 Sensor 클래스를 직접 작성
- 2 각 객체를 만들어 값 읽기
- 3 센서 정보를 dict로 변환해 보기



Day 3 10 상속으로 센서 확장하기

공통 기능은 부모 클래스에, 다른 기능은 자식 클래스에 작성합니다.

상속 개념

Sensor (부모 클래스)

- 공통 속성 : name, unit, value
- 공통 메서드 : read(), show(), to_dict()

TemperatureSensor



공통 기능을 상속받고 read()를 재정의하거나 확장합니다.

HumiditySensor



공통 기능을 상속받고 read()를 재정의하거나 확장합니다.

LightSensor



공통 기능을 상속받고 read()를 재정의하거나 확장합니다.



상속을 사용하면 공통 코드를 재사용하고, 센서별 동작만 따로 구현할 수 있습니다.

</> 코드 예시

```
class Sensor:
    def __init__(self, name, unit):
        self.name = name
        self.unit = unit
        self.value = 0

class TemperatureSensor(Sensor):
    def __init__(self, name):
        super().__init__(name, "°C")
    def read(self):
        self.value = 24.3
        return self.value
```

```
t = TemperatureSensor("Temp")
t.read()
t.show()
```

센서 확장 예시

TemperatureSensor



단위: °C
예: 24.3

HumiditySensor



단위: %
예: 58

LightSensor



단위: lux
예: 820



핵심 정리

- 1 부모 클래스는 공통 속성과 메서드를 제공
- 2 자식 클래스는 필요한 기능을 추가/수정
- 3 `super().__init__()`로 부모 초기화 재사용



실습 미션

- 1 TemperatureSensor, HumiditySensor, LightSensor 클래스를 만들어 보세요.
- 2 각 클래스에서 read() 결과가 다르게 나오도록 작성해 보세요.

11 dataclass 활용

데이터 저장용 객체는 dataclass로 간결하게 만들 수 있습니다.



Point

dataclass를 사용하면 `__init__`, `__repr__` 등을 자동으로 생성해 줍니다.

dataclass 기본 사용

```
from dataclasses import dataclass
from datetime import datetime

@dataclass
class SensorData:
    timestamp: datetime
    name: str
    value: float
    unit: str
```



@dataclass 데코레이터는 `__init__`, `__repr__` 등을 자동으로 생성해 줍니다.

사용 예시

```
from datetime import datetime

data = SensorData(
    timestamp=datetime.now(),
    name="Temp",
    value=23.5,
    unit="°C"
)
```



실행 결과 (예시)

```
SensorData(timestamp=2024-05-20 14:30:15.123456,
name='Temp', value=23.5, unit='°C')
```

dataclass의 장점



코드 간결화

`__init__`, `__repr__` 등을 직접 작성할 필요 없음



가독성 향상

어떤 데이터를 저장하는 객체인지 한눈에 파악 가능



유지보수 용이

필드 추가/수정 시 자동으로 반영

고급 옵션 (자주 사용)

기본값 설정

```
value: float = 0.0
```

타입 변환

```
from __future__ import annotations
```

불변 객체

```
@dataclass(frozen=True)
```

일부 필드 제외

```
from dataclasses import field
temp: float = field(repr=False)
```



객체를 dict/JSON/CSV 형태로 변환하여 파일에 저장할 수 있습니다.



Point

객체를 파일에 저장하려면 직렬화(serialize)가 필요합니다.
다시 불러올 때는 역직렬화(deserialize)를 합니다.

변환 흐름도



코드 예시

```

from dataclasses import asdict
import json, csv

data = asdict(data_obj)
json_str = json.dumps(data)

with open('log.csv', 'a', newline='') as f:
    writer = csv.DictWriter(f,
        fieldnames=["timestamp", "name", "value", "unit"])
    writer.writerow(data)
  
```

CSV 파일 예시 (log.csv)

```

timestamp,name,value,unit
2024-05-20 14:30:15.123456,Temp,23.5,°C
2024-05-20 14:30:20.654321,Humi,45.2,%
2024-05-20 14:30:25.111111,Light,1023,lux
...
  
```



IoT 시스템에서는 데이터를 파일이나 DB에 저장 가능 → 추후 분석, 시각화, 모니터링에 활용됩니다.

객체 저장 파이프라인



1. 객체 생성
dataclass 객체 생성



2. dict 변환
asdict()로 변환



3. 직렬화
JSON 문자열 생성



4. 파일 저장
CSV/JSON 파일로 저장



5. 활용
분석, 시각화, 모니터링

센서 데이터 읽거나 파일 입출력 등에서 발생할 수 있는 예외를 처리하여 프로그램이 종료되지 않고 안전하게 동작하도록 합니다.



Point

예외 처리를 하면 예상치 못한 오류가 발생해도 프로그램이 안정적으로 실행을 유지할 수 있습니다.

예외가 발생할 수 있는 상황



센서 읽기 실패

- 연결 문제, 센서 오류 등



파일 읽기/쓰기 실패

- 파일이 없거나 권한 문제 등



계산 오류

- 0으로 나누기 등

try - except 기본 구조

```
try:
    # 예외가 발생할 수 있는 코드
    data = sensor.read()
except Exception as e:
    # 예외 발생 시 실행할 코드
    print(f"오류 발생: {e} ")
else:
    # 예외가 없을 때 실행할 코드
    process(data)
finally:
    # 항상 실행할 코드(선택)
    sensor.close()
```

센서 읽기 예제

```
class Sensor:
    def read(self):
        # 실제 센서 읽기 코드 (여기서는 예외 발생 가정)
        raise IOError("센서 연결 실패")

sensor = Sensor()
try:
    value = sensor.read()
    print(f"센서 값: {value}")
except IOError as e:
    print(f"센서 오류: {e} ")
finally:
    print("센서 자원 해제")
```

다양한 예외 처리 예시

예외 유형	설명
IOError	입출력 관련 오류 (파일, 센서 등)
ValueError	잘못된 값이 입력되었을 때
ZeroDivisionError	0으로 나누기를 시도할 때
Exception	모든 예외의 기본 클래스



핵심 정리

- ✓ 예외를 예측하고 처리하면 프로그램이 더 견고해집니다.
- ✓ try 블록에는 예외가 발생할 수 있는 코드를 작성합니다.
- ✓ except 블록에서 예외 유형을 지정하고 처리합니다.
- ✓ else는 예외가 없을 때, finally는 항상 실행됩니다.

☆ 실전 팁

- 예외 메시지(e)를 출력하면 문제 원인 파악에 도움이 됩니다.
- 구체적인 예외 유형을 처리하면 더 정밀한 대응이 가능합니다.
- 센서/파일 자원은 finally에서 반드시 해제하세요.

센서 데이터는 현실에서 다양한 형태로 들어올 수 있습니다.
유효한 데이터인지 확인하고, 잘못된 데이터는 처리하는 것이 중요합니다.

검사해야 할 항목



값의 범위 확인

- 온도: -50 ~ 150 °C
- 습도: 0 ~ 100 %
- 조도: 0 ~ 100000 lux

NaN

NaN(숫자 아님) 확인

- 계산 불가능한 값 필터링

#

자료형 확인

- 숫자(float/int) 형태인지 확인



필수 값 확인

- 누락되거나 빈 값 처리

검증 함수 예시

```
def is_valid_temperature(value):
    if value is None:
        return False, "값이 없습니다."
    if not isinstance(value, (int, float)):
        return False, "숫자가 아닙니다."
    if value < -50 or value > 150:
        return False, "온도 범위를 벗어났습니다."
    if value != value: # NaN 확인
        return False, "NaN 값입니다."
    return True, None
```



사용 예시

```
ok, msg = is_valid_temperature(temp)
if not ok:
    print(f"온도 데이터 오류: {msg}, 값={temp}")
    # 오류 처리 또는 기본값 사용
```



Point

데이터 유효성 검사를 통해 프로그램의 안정성과 신뢰성을 높일 수 있습니다.

여러 센서 검증 예시

```
def validate_sensor_data(data):
    errors = []
    checks = [
        ("temperature", is_valid_temperature, "°C"),
        ("humidity", is_valid_humidity, "%"),
        ("light", is_valid_light, "lux"),
    ]

    for name, func, unit in checks:
        ok, msg = func(data.get(name))
        if not ok:
            errors.append(f"{name}: {msg}")

    return errors
```

결과 예시

```
errors = validate_sensor_data(data)
if errors:
    for e in errors:
        print(f"[데이터 오류] {e}")
else:
    print("모든 데이터가 정상입니다.")
```

적용 팁

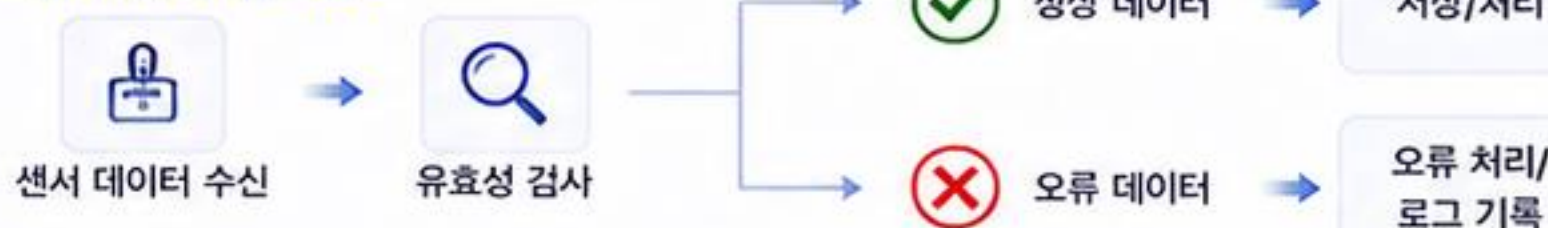
- ✓ 센서로부터 받은 값은 항상 검증 후 사용하세요.
- ✓ 오류가 발생해도 전체 프로그램이 중단되지 않도록 예외 처리하세요.
- ✓ 필요하면 기본값(default) 또는 이전 값(last value)을 사용하세요.



확장 아이디어

- 검증 로직을 별도 모듈로 분리하여 재사용하기
- 로그 파일에 검증 실패 내역 기록하기
- 연속된 오류가 발생하면 알림 보내기(경고 시스템)

데이터 처리 흐름



프로그램 실행 중 주요 정보를 로그로 남기면 문제 발생 시 원인 파악과 디버깅이 쉬워집니다.



Point

적절한 로그 기록은 프로그램의 가시성을 높이고 유지보수를 쉽게 해줍니다.

1. logging 기본 설정

```
import logging
from datetime import datetime

# 로그 설정
logging.basicConfig(
    filename='app.log',      # 로그 파일
    level=logging.INFO,      # 로그 레벨
    format='%(asctime)s [%(levelname)s] %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S'
)
logger = logging.getLogger(__name__)
```



로그 레벨

DEBUG < INFO < WARNING < ERROR < CRITICAL
레벨이 높을수록 중요한 메시지만 기록됩니다.

2. 로그 사용 예시

```
def process_sensor_data(data):
    logger.info(f"센서 데이터 수신: {data}")
    try:
        value = float(data['value'])
        logger.debug(f"변환된 값: {value}")
        # 데이터 처리 로직
        logger.info("데이터 처리 성공")
        return value
    except (ValueError, KeyError) as e:
        logger.error(f"데이터 처리 오류: {e}",
                    exc_info=True)
        return None
```

사용 예시

```
data = {'value': '23.5'}
result = process_sensor_data(data)
```

3. 로그 파일 예시 (app.log)

```
2024-05-20 14:30:15 [INFO] 센서 데이터 수신: {'value': '23.5'}
2024-05-20 14:30:15 [DEBUG] 변환된 값: 23.5
2024-05-20 14:30:15 [INFO] 데이터 처리 성공
2024-05-20 14:31:02 [INFO] 센서 데이터 수신: {'value': 'abc'}
2024-05-20 14:31:02 [ERROR] 데이터 처리 오류: could not convert
string to float: 'abc'
Traceback (most recent call last):
  File "sensor_app.py", line 25, in process_sensor_data
    value = float(data['value'])
ValueError: could not convert string to float: 'abc'
```

로그 활용 팁

- ✓ INFO: 주요 흐름 기록 (시작/종료, 성공 상황)
- ✓ WARNING: 주의가 필요한 상황 기록
- ✓ ERROR/CRITICAL: 오류 원인과 스택 트레이스 기록
- 📌 실서비스에서는 로그를 별도 시스템(파일 롤링, 로그 서버)에 저장하세요.

추가 팁



파일 롤링

로그 파일이 커지면 자동으로 분할
(backupCount, maxBytes 사용)



불필요한 로그 필터링

민감 정보는 기록하지 않도록 주의
(개인정보, 비밀번호 등)



로그 모니터링

로그를 주기적으로 확인하고
알림 설정으로 이상 감지

파일 롤링 설정 예시

```
from logging.handlers import RotatingFileHandler
handler = RotatingFileHandler('app.log', maxBytes=10*1024*1024,
                              backupCount=5, encoding='utf-8')
logging.getLogger().addHandler(handler)
```


수집한 센서 데이터를 그래프로 시각화하면
변화 추세와 이상치를 쉽게 발견할 수 있습니다.



Point

시각화는 데이터의 패턴을 빠르게 파악하고
문제 원인을 분석하는 강력한 도구입니다.

1. 시각화 코드 예시

```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

# CSV 파일 읽기
df = pd.read_csv('log.csv', parse_dates=['timestamp'])

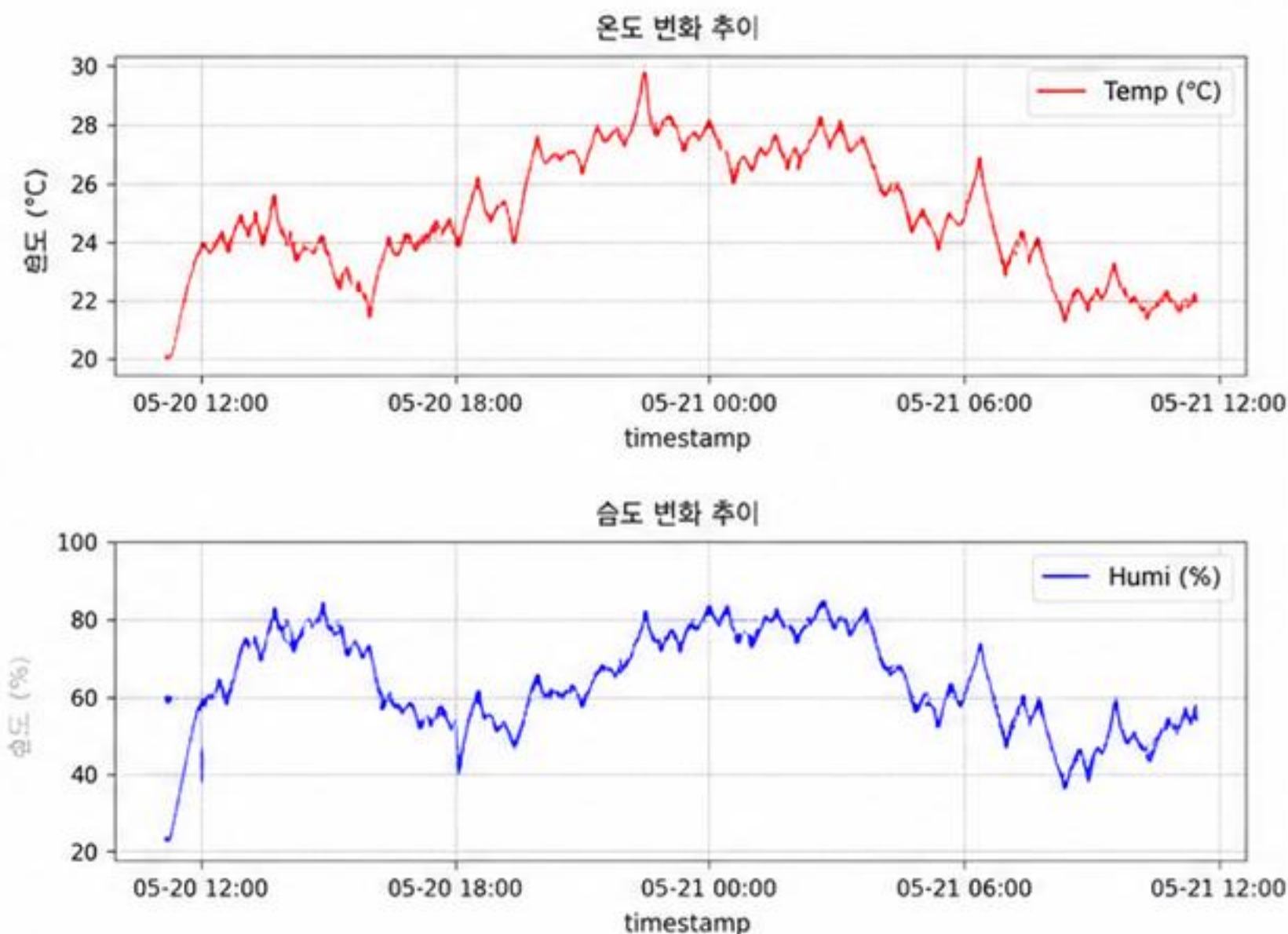
# 그래프 설정
plt.figure(figsize=(12, 6))

# 온도 그래프
plt.subplot(2, 1, 1)
plt.plot(df['timestamp'], df['Temp'], color='red', label='Temp (°C)')
plt.ylabel('온도 (°C)')
plt.title('온도 변화 추이')
plt.legend()
plt.grid(True)

# 습도 그래프
plt.subplot(2, 1, 2)
plt.plot(df['timestamp'], df['Humi'], color='blue', label='Humi (%)')
plt.ylabel('습도 (%)')
plt.title('습도 변화 추이')
plt.legend()
plt.grid(True)

# x축 포맷 설정 및 레이아웃 조정
plt.gcf().autofmt_xdate()
plt.tight_layout()
plt.show()
```

2. 실행 결과 예시



3. 확인할 포인트



변화 추세 파악

시간에 따른 온도/습도의 변화를
한눈에 확인



이상치 탐지

갑작스러운 급변이나 비정상 값
쉽게 발견



주기성 분석

일별/주별 패턴을 파악하여
원인 분석에 활용



데이터 품질 확인

결측값, 중복, 센서 오류 구간을
시각적으로 점검



시각화 팁

이동평균선을 추가하면 장기 추세를 더 쉽게 파악할 수 있습니다.

이상치 구간을 다른 색으로 표시하면 문제 구간을 빠르게 인지할 수 있습니다.

여러 센서를 한 그래프에 비교하면 상관관계 분석이 가능합니다.

정기적으로 그래프를 확인하는 습관이 문제 예방의 핵심입니다.

이동평균선 예시

```
df['Temp_MA'] = df['Temp'].rolling(window=10).mean()
plt.plot(df['timestamp'], df['Temp_MA'],
         color='orange', label='Temp MA(10)')
```


수집한 센서 데이터를 분석하여 이상치를 탐지하고,
발생 원인을 가설로 세워 데이터로 검증하는 방법을 알아봅니다.



Point

이상치는 단순한 오류가 아니라 시스템 이상 신호일 수 있습니다.
데이터 기반으로 원인을 추론하고 검증하는 것이 중요합니다.

1. 이상치 탐지 (IQR 방법)

```
import pandas as pd
import numpy as np

# 온도 데이터
temp = df['Temp']

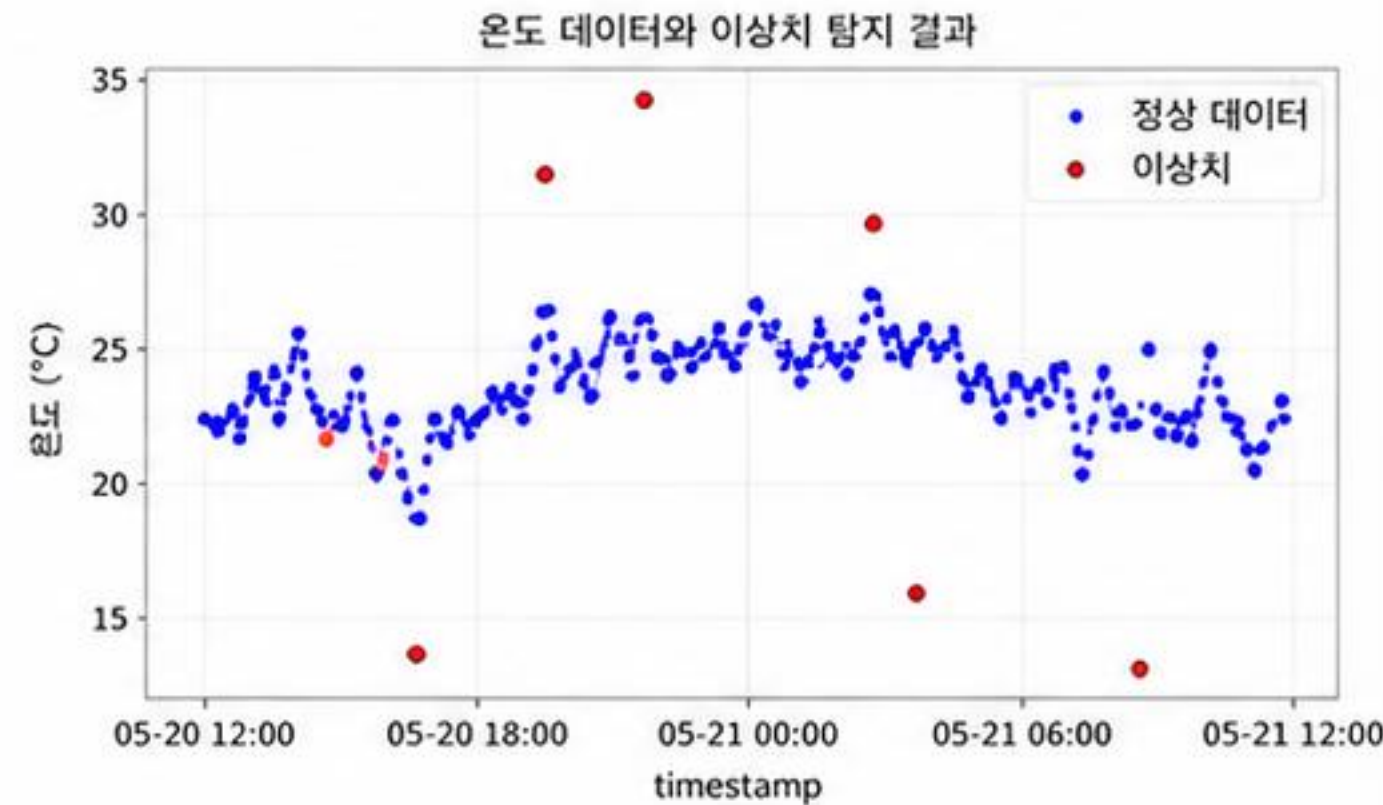
# IQR 계산
Q1 = temp.quantile(0.25)
Q3 = temp.quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# 이상치 추출
outliers = df[(temp < lower) | (temp > upper)]
print(f"이상치 개수: {len(outliers)}건")
```

IQR(사분위 범위) 기준

- lower 미만 또는 upper 초과 값을 이상치로 판단
- 분포의 영향을 덜 받아 안정적으로 이상치 탐지 가능

2. 이상치 시각화 예시



시각화 포인트

- 빨간 점: IQR 기준 이상치
- 특정 시간대에 이상치가 집중되었는지 확인

3. 원인 분석 가설 수립 및 검증

단계	내용	예시
가설 수립	이상치 발생 원인을 가설로 설정	"오전 2시 경 냉방 장치 이상 동작"
관련 데이터 확인	해당 시간대의 다른 센서 데이터를 확인	습도, 조도, 장치 상태 등 함께 확인
비교 분석	이상치 구간과 정상 구간을 비교하여 차이점 분석	온도 급상승 + 습도 급하락 패턴 발견
결론 도출	데이터 기반으로 원인 여부 판단	냉방 장치 재가동 기록과 일치 → 원인 가능성 높음

예시: 온도 이상치 원인 분석

- 이상치 발생 시간: 05-21 02:00 ~ 03:00
 - 습도: 급격히 감소 (정상: 60% → 이상 구간: 30%)
 - 장치 로그: 02:01 냉방 장치 재가동 기록 확인
- ⇒ 결론: 냉방 장치 이상 동작으로 인한 온도 급상승 가능성 높음

추가 분석 방법



Z-Score 방법

평균과 표준편차 기반
이상치 탐지



이동평균 기반 탐지

이동평균에서 벗어난
값을 이상치로 판단



도메인 지식 활용

센서 특성 및 시스템 이해를
통해 정교한 판단

분석 시 유의사항

- ✓ 센서 오류와 실제 이상 현상을 구분하여 판단합니다.
- ✓ 데이터 전처리(결측치 처리, 평활화) 후 분석하면 정확도가 높아집니다.
- ✓ 이상치가 반복적으로 발생하면 시스템 점검이 필요합니다.

활용 방안



- 시스템 이상 감지 및 알람 연동
- 예방 유지보수(Predictive Maintenance)
- 데이터 품질 개선 및 신뢰성 향상

이상치는 분석 결과에 큰 영향을 줄 수 있습니다.

상황에 맞는 전략을 선택하여 이상치를 적절히 처리하는 방법을 알아봅니다.



Point

이상치는 무조건 삭제하는 것이 아니라,
데이터의 특성과 분석 목적에 맞게 처리해야 합니다.

1. 이상치 처리 전략



제거 (Delete)

명백한 오류거나 분석에 불필요한 경우
해당 데이터를 삭제합니다.



대체 (Impute)

중앙값, 평균, 분위수 등으로 대체하여
데이터의 손실을 최소화합니다.



원저화 (Winsorize)

이상치 값을 경계값으로 대체하여
영향을 줄이지만 데이터를 유지합니다.



변환 (Transform)

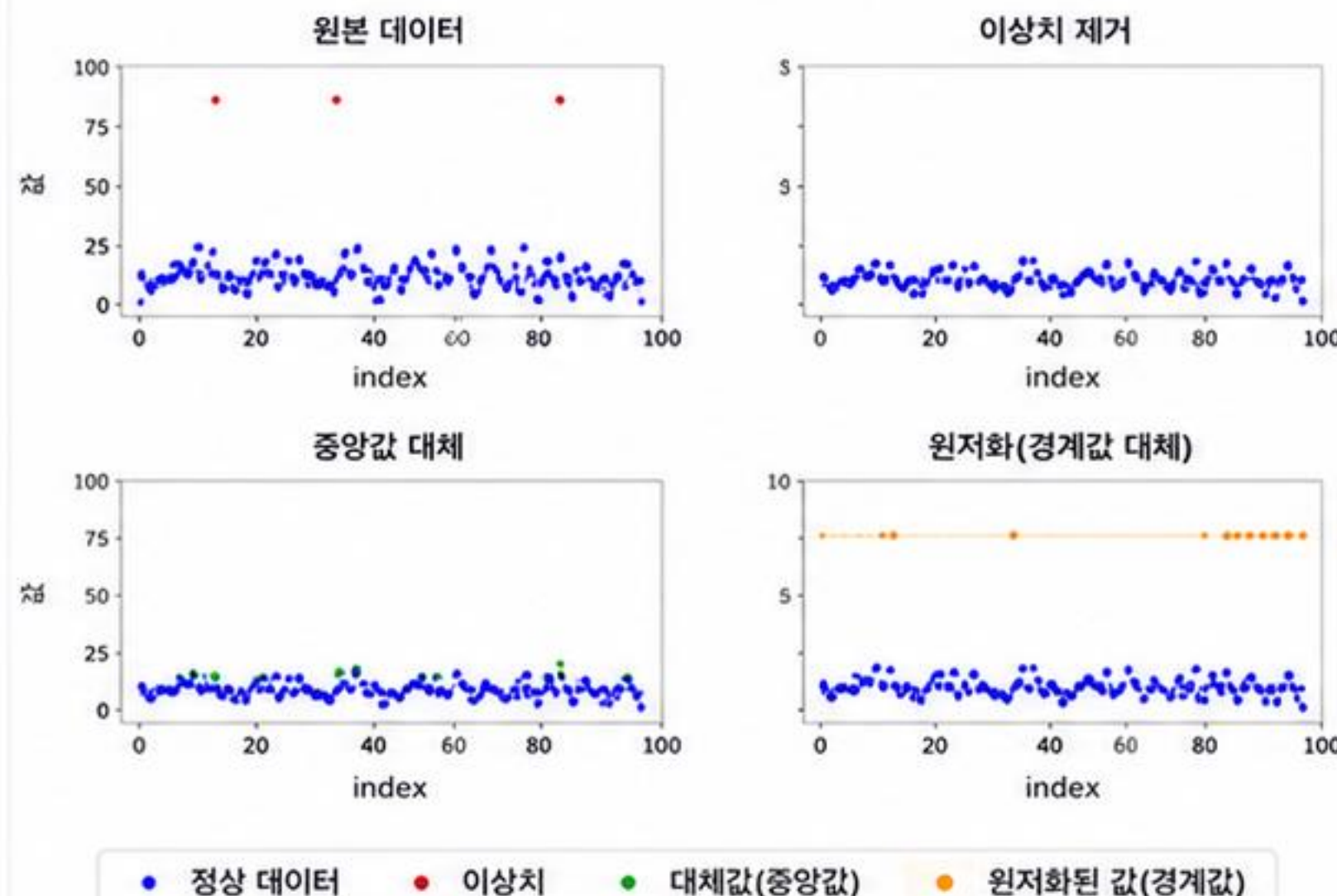
로그 변환, 제곱근 변환 등을 통해
이상치의 영향을 완화합니다.



플래그 지정 (Flagging)

이상치 여부를 표시만 하고,
원본 데이터를 그대로 유지합니다.

2. 예시 데이터와 이상치 처리 효과



3. 파이썬 코드 예시

```
import pandas as pd
import numpy as np

# 예시 데이터 생성
np.random.seed(42)
data = pd.DataFrame({'value': np.random.normal(10, 2, 100)})

# 이상치 추가
data.loc[[5, 20, 55], 'value'] = [50, 60, 80]

# 1) 제거 (IQR 방법)
Q1 = data['value'].quantile(0.25)
Q3 = data['value'].quantile(0.75)
IQR = Q3 - Q1
lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR
df_remove = data[(data['value'] >= lower) & (data['value'] <= upper)]

# 2) 중앙값 대체
median = data['value'].median()
df_impute = data.copy()
df_impute.loc[(data['value'] < lower) | (data['value'] > upper), 'value'] = median

# 3) 원저화
df_winsor = data.copy()
df_winsor['value'] = df_winsor['value'].clip(lower, upper)
```

4. 어떤 방법을 선택할까?

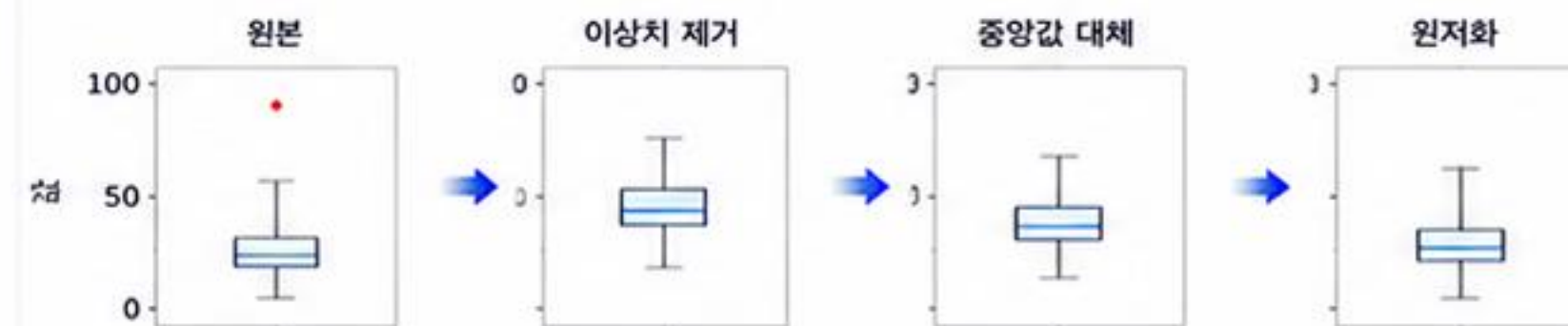
상황	추천 방법
명백한 측정 오류	제거
데이터 손실이 민감한 경우	대체 (중앙값/평균)
이상치가 극단값이지만 유의미한 경우	원저화 또는 변환
이상치 여부만 확인하고 싶을 때	플래그 지정

5. 주의사항



- 무조건 삭제하면 데이터의 분포가 왜곡될 수 있습니다.
- 처리 전후로 데이터 분포와 통계량을 반드시 비교하세요.
- 도메인 지식을 고려하여 이상치가 중요한 정보일 수 있습니다.
- 분석 목적에 따라 다른 전략을 적용하는 것이 좋습니다.

6. 간단한 분포 비교



수집한 센서 데이터의 전반적인 통계 정보를 요약하면
데이터의 분포와 특성을 빠르게 파악할 수 있습니다.



Point

통계 요약은 데이터의 중심 경향, 산포, 이상치 등을
한눈에 보여주어 데이터 이해와 의사결정에 도움을 줍니다.

1. 통계 요약 코드 예시

```
import pandas as pd

# 센서 데이터 불러오기
df = pd.read_csv('sensor_clean.csv')

# 수치형 컬럼 선택
num_cols = df.select_dtypes(include=['number']).columns

# 기술 통계 요약
desc = df[num_cols].describe()

# 추가 통계: 왜도, 첨도
skew = df[num_cols].skew()
kurt = df[num_cols].kurtosis()

# 결측치 개수
missing = df.isnull().sum()

# 결과 출력
print("> 기술 통계 요약\n", desc.round(2))
print("\n> 왜도(Skew)\n", skew.round(2))
print("\n> 첨도(Kurtosis)\n", kurt.round(2))
print("\n> 결측치 개수\n", missing)
```

2. 실행 결과 예시

▶ 기술 통계 요약

	temperature	humidity	light	lux
count	1440.00	1440.00	1440.00	1440.00
mean	23.45	56.78	312.14	1523.67
std	2.18	9.63	120.45	812.33
min	18.20	30.00	10.00	5.00
25%	21.90	50.00	220.00	880.00
50%	23.50	57.00	305.00	1420.00
75%	25.00	63.00	400.00	1960.00
max	30.10	85.00	850.00	3580.00

▶ 왜도(Skew)

temperature 0.12 humidity -0.21 light 0.68 lux 1.05

▶ 첨도(Kurtosis)

temperature -0.18 humidity -0.32 light 0.91 lux 1.85

▶ 결측치 개수

temperature 0 humidity 0 light 0 lux 0

3. 주요 통계 지표 설명



count

데이터 개수



mean

평균(산술 평균)



std

표준편차(데이터 산포도)



min / 25% / 50% / 75% / max

최솟값, 사분위수, 최댓값(분포 파악)



skew(왜도)

분포의 비대칭 정도 (양수: 오른쪽 꼬리 / 음수: 왼쪽 꼬리)



kurtosis(첨도)

분포의 뾰족한 정도 (양수: 뾰족 / 음수: 평평)



결측치 개수

데이터 품질을 확인하는 기본 지표

분석 팁

- ✓ 여러 센서의 통계를 비교하면 각 센서의 특성을 이해할 수 있습니다.
- ✓ 이상치 탐지 전, 통계를 통해 분포 범위를 가능해 보세요.
- ✓ 시간대별/일별로 그룹화하여 통계를 내면 패턴 파악에 유용합니다.



확장 아이디어

- 시간대별 그룹 통계: `df.groupby(df['hour']).mean()`
- 상관관계 분석: `df.corr()`
- 히스토그램으로 분포 시각화
- 자동 리포트 생성 (HTML/Excel)

주의사항

- ⚠ 센서 오류나 결측치 많으면 통계가 왜곡될 수 있습니다.
- ⚠ 단위가 다른 센서끼리는 비교에 주의하세요.
- ⚠ 극단값이 평균/표준편차에 큰 영향을 줄 수 있습니다.

지금까지 배운 내용을 종합하여 센서 데이터를 분석하고, 이상치를 탐지하며 시각화 보고서까지 작성해봅니다.



Point

데이터 분석은 흐름이 중요합니다. 수집 → 전처리 → 탐지 → 분석 → 시각화 → 해석의 과정을 기억하세요!

1. 전체 분석 코드 흐름

```
# 1. 데이터 로드
df = pd.read_csv('sensor_clean.csv')

# 2. 시간 인덱스 설정
df['timestamp'] = pd.to_datetime(df['timestamp'])
df = df.set_index('timestamp').sort_index()

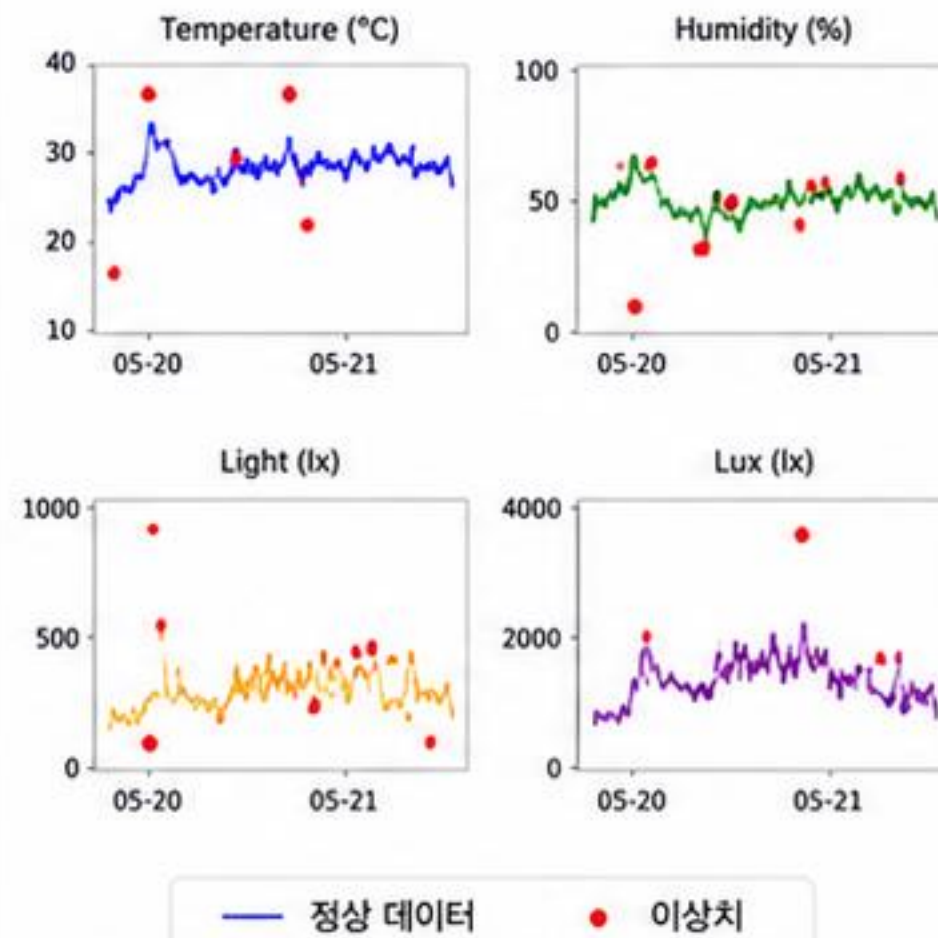
# 3. 이상치 탐지 (IQR)
def detect_outliers_igr(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    return data[(data[column] < lower) | (data[column] > upper)]

# 4. 각 센서 이상치 탐지
outliers = {}
for col in ['temperature', 'humidity', 'light', 'lux']:
    outliers[col] = detect_outliers_igr(df, col)

# 5. 시각화
plot_dashboard(df, outliers)

# 6. 요약 통계 출력
summary = df.describe().T
print(summary.round(2))
```

2. 분석 결과 예시



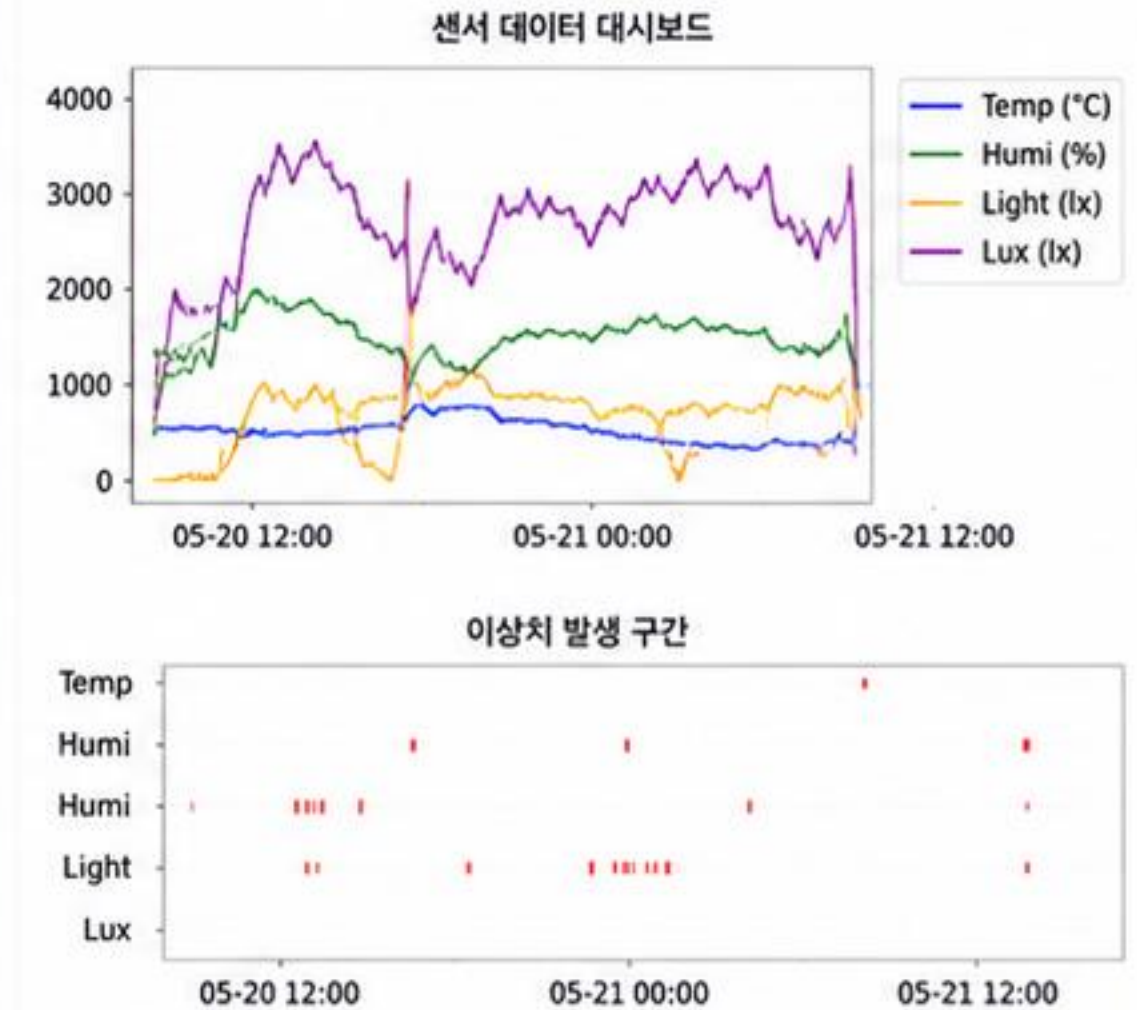
3. 요약 통계 예시

변수	count	mean	std	min	25%	50%	75%	max
temperature	1440	23.45	2.18	18.20	21.90	23.50	25.00	30.10
humidity	1440	56.78	9.63	30.00	50.00	57.00	63.00	85.00
light	1440	312.14	120.45	10.00	220.00	305.00	400.00	850.00
lux	1440	1523.67	812.33	5.00	880.00	1420.00	1960.00	3580.00

해석 예시

- 온도: 평균 23.45°C, 최대 30.10°C로 안정적
- 습도: 평균 56.78%, 일부 구간에서 급변
- 조도/조도(lux): 특정 시간에 급격한 증가 패턴 확인
- 이상치는 주로 온도와 조도에서 발생

4. 시각화 대시보드 예시



분석 체크리스트

- ✓ 데이터 로드 및 시간 인덱스 확인
- ✓ 결측치 처리 및 기본 통계 확인
- ✓ 이상치 탐지 및 개수 확인
- ✓ 시각화를 통한 패턴 파악
- ✓ 결과 해석 및 보고서 작성



보고서 작성 예시

분석 기간: 2024-05-20 ~ 2024-05-21
 총 데이터 수: 1,440건 (1분 간격)
 이상치 총 개수: 23건
 - 온도: 5건, 습도: 7건, 조도: 6건, 조도(lux): 5건
 주요 발견 사항:
 - 야간 시간대에 조도(lux) 급증 현상 발생
 - 특정 구간(05-21 02:00~03:00) 온도 이상치 집중
 권장 조치: 센서 위치 재점검 및 외부 요인 확인



추가 분석 아이디어

- 시간대별/요일별 패턴 분석
- 상관관계 분석 (예: 온도 vs 습도)
- 이동평균을 활용한 추세 분석
- 예측 모델(ARIMA, Prophet) 적용
- 이상치 알림 시스템 구축

주의사항

- ⚠ 이상치는 항상 오류가 아닐 수 있습니다.
- ⚠ 도메인 지식을 바탕으로 해석하세요.
- ⚠ 센서 교체/점검 이력과 함께 분석하세요.
- ⚠ 단일 지표만으로 판단하지 말고 종합적으로 분석하세요.



다음 단계

Day 4에서는
실시간 데이터 처리와
알림 시스템 구축을
학습합니다!